Flow

A High Performance Spark Data Processing Library

Chris Twiner

Table of contents

1. Flow - 0.0.1-poc8 3
   1.1 Process a Graph of data processing Steps with Quality 3
   1.2 Flow and FlowT? 3
   1.3 Serialisation 4
   1.4 Timeouts and Error handling 4
2. About 6
   2.1 History 6
   2.2 Changelog 7
3. Scala Docs 8

1. Flow - 0.0.1-poc8

verage			
Statement	96.48 %	Branch	90.74 %

1.1 Process a Graph of data processing Steps with Quality

Write auditable DQ and transformation rules using simple SQL or create re-usable functions via SQL Lambdas and manage them through a graph of Steps.

Your Flows can be provided DataFrames from either a Quality DataFrameLoader, catalog or even custom loading logic via extension of FlowT.

- 🆕 Initial Release

Entire Flows are lazily evaluated but support caching of a Steps results for interim performance benefits.

Each Step in a Flow has an associated Runner and, unless configured not to, keeps an audit of all the results of the previous Steps in the graph, either Quality:

1. engine,
2. collector,
3. folder,
4. DQ or
5. a custom implementation

The resulting Column then has a ResultApproach applied to it, either:

- AsIs - the audit and result column is added to the DataFrame,
- MergeFields - any nested result fields are merged with the DataFrame,
- OutputFieldsOnly - only the audit column and nested result columns are output,
- StarOnly - the audit column is dropped and only nested result columns are output,
- OutFieldOnly - only the result column remains or
- a custom implementation

1.2 Flow and FlowT?

The default Flow is a type alias for:

```
type Flow = FlowT[CombinedRuleSuiteRows, RuleSuite]
```

The first parameter is for the type of Flow / "group level" rules, these may be accessed as nested runners within transformations, the second parameter is the type of RuleSuiteTypeParam parameter to use, which are passed to the Runner functions.

Flow supports both GroupRuleId and RuleSuite RuleSuiteTypeParam in Steps *and* also Id's at group level, with the exact types used following through to serialisation.

1.2.1 What is returned?

Flow's Steps are processed as graphs of Futures, each effectively returning a DataFrame, which can be optionally cached, forming the inputs to the next dependent Step. These Futures are then sequenced to return a FlowResult:

```
case class FlowResult[T: RuleSuiteTypeParam](stepResults: Map[String, StepResultType[T]],
                                             duration: Duration) extends Serializable
```

wherein each StepResultType is either a StepResult or a StepException, by default any return is only StepResults, use the tolerant parameter to decide how to handle failure.

```
case class StepResult[RP: RuleSuiteTypeParam](step: Step[RP], output: DataFrame, timings: StepTimings)
```

StepResult itself provides the resulting DataFrame, as the Flow can have multiple final leaf nodes the caller must know what to do with them, for example custom ResultApproaches may save the DataFrame to disk, or the Flow.run caller has custom knowledge of the flows.

1.3 Serialisation

Flow supports both individual datasets for a more normalised flow management approach and a FullFlow approach allowing individual flows to be serialised via json, for example. Although the default behaviour of both Flow's and serialisation is group level CombinedRuleSuites and RuleSuites, the types can be converted to group level rules and GroupRuleId's via the convertToIds function.

As the Id's and Rules can be sourced from different locations in a Flow there are a number of strategies that can be provided to serialisation functions:

- RuleSuiteFromDataset
- RuleSuiteFromFlows
- IdFromFlows
- IdFromDatasets

these customise the processing of configuration data.

Serialising the default Flow to a FullFlow will yield a RuleSuite at each Step, doing so with GroupId's as the Step type just has the ruleSuiteId and ruleSuiteVersion saved.

Flow makes use of the sparkutils fork of Frameless to use AgnosticEncoding for Spark 4+ for correct type handling, this also allows deciding on which type of FlowT should be used to be abstracted and combined by Frameless' encoder derivation approach.

1.4 Timeouts and Error handling

1.4.1 Timeouts

By default, a Flow will run until normal completion, but in many cases Flows will be part of business processes with tight SLAs where detecting errand processes early is important. As such there are two layers of timeouts..

1. Flow level - configured via FlowT parameters and FlowRow "duration" fields.
2. Step level - configured via a Step's stepTimeout options

Step level timeouts can operate independently, allowing a given problematic step to be configured with a timeout that then applies to the whole Step sub-graph.

When a timeout is reached a StepException is returned for a Step instead of a StepResult.

1.4.2 Exception / Error handling

When calling Flow.run the default value of the "tolerant" parameter is 'false' e.g.:

```
val ires = flow.run(sparkSession)
```

In this default behaviour **any** exception, whether in Spark or via stepTimeout options, will trigger the run call to throw.

In contrast:

```
val ires = flow.run(sparkSession, tolerant = true)
```

will return a mix of responses for each Step. You can use either pattern matching or the StepResultType.fold function to process failures and successes.

Last update: June 7, 2026 17:59:52
Created: June 7, 2026 17:59:52

2. About

2.1 History

2.1.1 Why Flow?

In short, the need for a lightweight configurable and auditable transformation approach integrated within a Spark application wasn't being met by other identified solutions. Many full-fledged products required additional infra, or required using Python instead of configuration through data suitable for audit and end-user maintenance.

Flow aims to fill the 'in process' data processing library gap configurable via simple sql snippets and some json.

Last update: June 7, 2026 17:59:52

Created: June 7, 2026 17:59:52

2.2 Changelog

0.0.1 Initial Release 1st May, 2026

The initial release of Flow supports the processing of a multi-root DAG of Steps, each performing a Quality transformation, or DQ check, of some data.

This release is built on Spark 4/4.1 builds of Quality 0.2.0, with the two Quality data models being supported (de-normalised and CombinedRuleSuiteRows) for rule maintenance allowing Flow to serialize to json configuration files suitable for publishing between development environments as well as a more gui friendly format.

Last update: June 7, 2026 17:59:52
Created: June 7, 2026 17:59:52

3. Scala Docs

flow_api docs.

The server side interface can be found here.

Last update: June 7, 2026 17:59:52

Created: June 7, 2026 17:59:52